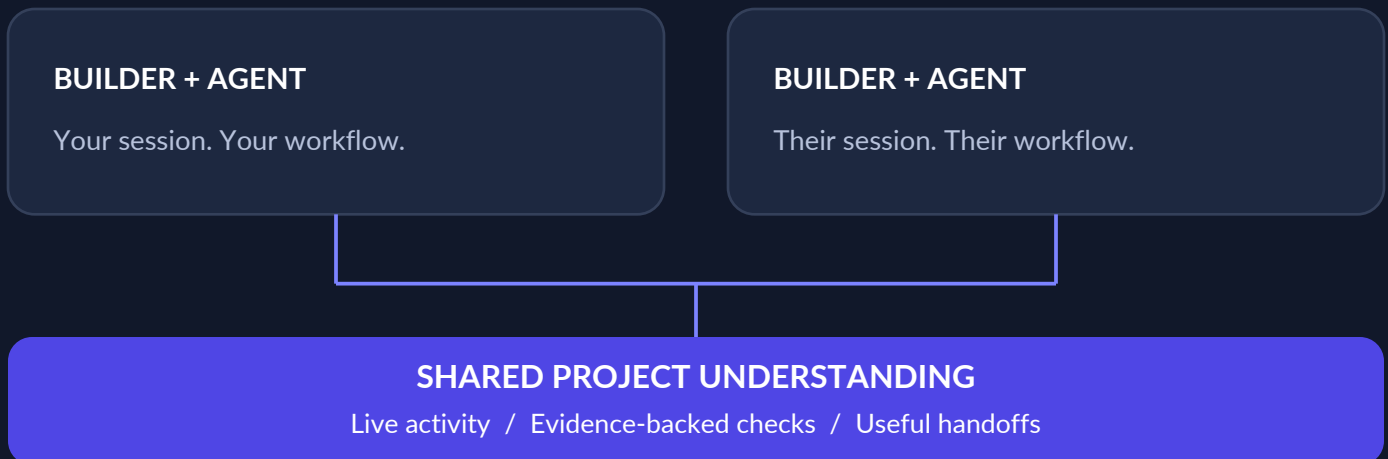


Build together. With your agents.

Builder Collaboration Workspace

A shared workspace that connects builders and their coding agents, makes team activity visible, and turns overlapping work into coordinated progress.



Read this first

A complete product definition with a deliberately narrow first release.

This document defines a **demoable, functional, well-designed collaboration app** - not just a transcript viewer. It preserves the broad product vision while making the first release implementable: two builders, one repository, one agent integration, and one complete coordination loop.

P0 = required for the demo release. P1 = improvement after the loop works. Targets and effort estimates are proposed, not measured. Numbered requirements and QA cases are intended to be used as the implementation checklist.

01 Product vision	3	18 Interaction and accessibility	20
02 Users and product principles	4	19 Technical architecture	21
03 Release scope	5	20 Data model	22
04 The end-to-end experience	6	21 API surface	23
05 Workspace and onboarding	7	22 Example data contracts	24
06 Agent connection and sharing	8	23 Adapter execution and continuation	25
07 Live work and active tasks	9	24 Security and privacy	26
08 Shared session viewer	10	25 Reliability and operations	27
09 Before-you-build checks	11	26 Metrics and evaluation	28
10 Jev decision system	12	27 Quality assurance matrix	29
11 Repository evidence and coverage	13	28 Build plan and gates	30
12 Coordination and context handoff	14	29 Live demo runbook	31
13 Concurrency and lifecycle rules	15	30 Definition of done	32
14 Information architecture	16	31 Risks and roadmap	33
15 Design direction and resources	17	32 Builder kickoff brief	34
16 Workspace composition	18	33 Sources and status	35
17 Critical screen specifications	19		

Recommended reading: product and scope (pages 3-6); design specification (pages 17-20); technical build (pages 21-27); implementation and demo gates (pages 30-34).

01 / Product vision

A shared workspace for builders and their coding agents.

Everyone keeps their own coding agent. The team gains a shared understanding of the project.

The product connects **builder-agent pairs**: a person and the coding agent they already use. It brings their shared project activity into one workspace and delivers relevant team context before an agent starts implementing a change. The outcome is coordinated building, not another place to read logs.

The problem we are solving

Our product hypothesis is that teams using separate agent sessions lose context between people: what is being built, what already exists, which decisions were made, and where work overlaps. The user has identified this workflow problem; its frequency and commercial demand still need validation. This PRD does not present unmeasured productivity claims as established facts.

The two core capabilities

Shared activity: teammates can inspect intentionally shared agent conversations, visible responses, tool calls, files, and task context. **Before-you-build coordination**: a Jev-backed check compares a proposed change with relevant code evidence and active team work, then offers evidence-backed ways to reuse, coordinate, or proceed.

The product promise

A builder should be able to answer "What is everyone doing?" in a few seconds. An agent should receive the relevant answer to "What should I know before I start?" inside its existing workflow. The workspace connects these two experiences without replacing the coding environment.

Why it fits the hackathon theme

For "Build something agents want," this app gives agents a useful capability: awareness of the people, agents, work, and decisions around them. The demo must show that context changing actual agent behavior, not merely producing a dashboard notification.

Positioning hypothesis: outcome-level coordination with evidence and useful handoffs. Do not claim that shared-agent feeds or overlap detection are new categories. No exhaustive competitor or trademark review is included.

02 / Users and product principles

Convenient collaboration, with human control.

User	Primary job and observable success
Builder	Understand teammates' work, avoid repeating it, and continue in the existing editor or terminal. A relevant check arrives without manually opening the dashboard.
Project owner	Create the shared project, invite teammates, and manage connections and access. Can see connection health without gaining access to private sessions.
Coding agent	Receive bounded, current context with source references. Can adjust its next task without searching everyone's entire history.

Design for these initial teams

The first release targets a small hackathon or side-project team: two to five builders, one shared repository, and one supported coding-agent adapter. This is a deliberate build boundary, not an assertion that larger organizations cannot use the concept.

Product principles

Work normally. Capture useful activity automatically after consent; do not turn every prompt into a project-management form. **Evidence over confidence theater.** Link to the exact task, message, or code snapshot behind a warning. **Assist, do not police.** Builders can disagree, revise, or proceed with an explicit override. **Sharing is intentional.** Connection does not mean uploading every local conversation. **Context is scoped.** Planned work, branch code, merged code, and stale evidence are different. **Coordination creates an action.** A warning should lead to reuse, a handoff, a clarified boundary, or a conscious decision to continue.

What the product is not

It is not a new coding model, IDE, remote execution service, automatic merge engine, generalized QA tool, employee surveillance system, or complete replacement for GitHub and project management. It does not promise that semantic overlap will always be detected.

Terminology: "transcript" means shareable user-visible messages and supported tool events. Hidden model reasoning, credentials, provider-internal state, and unrelated local sessions are outside the product's capture scope.

03 / Release scope

P0 is the functional demo. P1 improves the product after that loop works.

Priority	Committed scope
P0 / essential	Authentication; one organization and project; join by invitation; two connected users; one Claude Code adapter; explicit sharing; live activity; session viewer; active tasks; preflight checks; evidence-backed outcomes; resolution and context handoff; real deployment.
P0 / trust	Project isolation; private-session exclusion; local redaction; device revocation; status freshness; visible provider failures; tested event deduplication; no fictitious live data.
P1 / valuable	Saved interface contracts, better task search, richer source indexing, notification preferences, improved summaries, a second adapter only after the first is stable.
Later	Multiple projects in the UI; multiple repositories; GitHub App and PR sync; deeper code indexing; agent-to-agent negotiated handoffs; enterprise policies and billing.

Bounded code awareness

P0 checks a small TypeScript/JavaScript demo repository using an approved snapshot, a source inventory, and focused snippets. It must detect relevant existing implementation evidence in that repository. It must not claim to prove that a feature is completely implemented or absent across any arbitrary codebase.

Explicit non-goals

No embedded terminal, browser IDE, autonomous code execution on the server, general chat product, billing system, automatic PR merge, universal agent compatibility, or decorative analytics dashboard. Native mobile apps, continuous screen capture, and a public transcript directory are also excluded.

Default decisions to prevent planning stalls

Use Next.js, TypeScript, Supabase, Vercel, and a locally running adapter. Claude Code is the proposed first integration, subject to the first build gate proving interception and context return on the exact installed version. Jev is the classification provider. The product name remains undecided; this document uses "Builder Collaboration Workspace."

Scope changes need a stated user outcome and a removed or deferred item. Protect the live coordination loop before expanding the feature list. No hackathon duration, spending allowance, or code-reuse permission has been confirmed.

04 / The end-to-end experience

One coherent journey from connection to coordinated work.

Step	Experience and acceptance signal
1. Create	Sri signs in, creates a project, and gets an invitation link. Another authenticated user joins that exact project.
2. Connect	Each builder runs the adapter locally, pairs it with the project, selects the repository, and previews what will be shared. Both connections become visible.
3. Start	Sam asks their agent to implement the saved-college backend. A proposed task is registered before implementation and becomes active when Sam proceeds.
4. Observe	Sri sees the task title, owner, branch, status, and recent tool activity. The full session can be opened without losing the project view.
5. Check	Sri asks for a shortlist backend. The app checks current team intent and relevant code evidence before the agent begins implementing that request.
6. Resolve	A likely duplicate is shown with Sam's actual task and transcript evidence. Sri selects coordination and defines a complementary UI task.
7. Continue	Sri explicitly resubmits the revised task. The adapter supplies an approved context packet. The agent acknowledges the boundary and works on the UI.
8. Confirm	The workspace shows two distinct tasks, the coordination decision, and subsequent actual tool activity. It does not claim that either task is complete without evidence.

The convenience requirement

A clear check should be unobtrusive. Related work should appear as useful context. A likely duplicate should make the overlap understandable and provide a next step. Builders should not have to maintain a second task tracker just to obtain these benefits.

Demo success: two overlapping requests become two complementary pieces of real work.

PO does not promise automatic terminal resumption from a browser button. The supported flow uses an explicit resubmission with an approved resolution reference. This avoids pretending that the dashboard controls an arbitrary running agent.

05 / Workspace, membership and onboarding

Requirements FR-01 through FR-05.

FR-01 / Identity and workspace

Authenticate users before accessing real projects. Create an organization with a display name and one project with a name, short purpose, and repository identifier. Assign the creator as owner. Persist the selected project across reloads; never infer authorization from a route slug alone.

FR-02 / Invitations and membership

The owner can create, copy, and revoke a single-use invitation. Proposed default: seven-day expiry. Store only a token hash; show the plaintext invite only at creation. Accepting an invite requires authentication and an atomic token-consumption transaction. Expired, revoked, and already-used invites have distinct recovery messages.

FR-03 / Two simple roles

Action	Owner	Member
View shared project activity and evidence	Yes	Yes
Connect an agent; share or pause own sessions	Yes	Yes
Edit own task; resolve own check; contribute a handoff	Yes	Yes
Invite/remove members; revoke project devices	Yes	No
Read another user's private or unshared sessions	No	No

FR-04 / Guided connection

Use four visible steps: project selected, adapter installed, device paired, first event received. Each step has a success state and an actionable error. A connected badge appears only after an authenticated heartbeat, not merely after a command was copied.

FR-05 / Manage and leave

Members can disconnect devices, pause future sharing, delete their shared session content, and leave the organization. The owner can remove members but cannot transfer the last-owner role into an invalid state.

Deletion of an organization is separately confirmed and queued for cleanup.

Acceptance: two independent accounts can join the same project; an unrelated account cannot fetch its records; a revoked invitation cannot be reused; removing a member immediately denies subsequent authorized fetches and ingestion.

06 / Agent connection and sharing

Requirements FR-06 through FR-10.

FR-06 / One tested adapter first

Build a small local CLI/bridge, not a hosted agent runner. It binds a device to one user, project, and repository root; receives supported lifecycle events; uploads approved activity; and returns coordination context. Record agent and adapter versions in every session. Unknown versions trigger a compatibility warning, not silent best-effort capture.

Claude Code documents prompt-submission hooks before processing and tool hooks around execution. Its hook interface can also add context. These are the initial integration points; exact behavior must pass a local spike on the installed version. [S01]

FR-07 / Explicit consent

Default to not sharing. At connection, show the approved repository root, the destination project, message/tool categories, source-snippet policy, and provider disclosure. Let the user inspect a sanitized sample before enabling project sharing. Persist consent version and timestamp. A session has a visible sharing indicator and pause action.

FR-08 / Capture contract

Capture user-visible prompts and responses, tool name, sanitized arguments/result excerpt, status, timestamp, correlation ID, and relative files when supported. Capture only selected project sessions. Do not upload raw environment variables, hidden reasoning, home-directory paths, complete terminal history, or unrelated attachments.

FR-09 / Local safety and delivery

Apply path exclusions and redaction before queueing or uploading. Respect repository ignore rules plus an explicit denylist for secrets, private keys, generated folders, and large/binary files. Use a bounded local spool, deterministic event IDs, retry backoff, and a flush acknowledgement. Pausing sharing stops capture and clears unsent content unless the user explicitly retains it.

FR-10 / Connection health

Show connected, reconnecting, disconnected, unsupported, and revoked states. Proposed heartbeat interval: 15 seconds; stale after 60 seconds. These indicate device freshness, not whether a person is working. Reconnection resends unacknowledged events without duplication.

Acceptance: two real sessions appear independently; a known test secret never reaches the network payload; a disconnected device is not shown as live; revocation stops future uploads.

07 / Live work and active tasks

Requirements FR-11 through FR-15.

FR-11 / Project overview

Show the project identity, connection health, active builder-agent pairs, current work, recent meaningful activity, and unresolved checks. The first screen answers who is working on what. Raw event volume, token leaderboards, and empty KPI tiles are not primary UI.

FR-12 / Explicit task records

Register the implementation request as a proposed task before checking it. Store the owner, session, original request, editable title, intended scope, branch, base commit when available, lifecycle state, revision, and last activity. Use the original prompt as the source of truth. A shortened first line is an acceptable initial title; a summarizer must not invent scope.

FR-13 / Task lifecycle

State	Meaning
Proposed / checking	Intent has been registered; it is not yet evidence of active implementation.
Active	The builder proceeded and the session is performing the task.
Needs coordination / paused	A decision or dependency is pending, or the owner paused work.
Awaiting review	The agent reported completion; not independently verified as complete.
Done / cancelled	The owner confirmed completion or abandoned the task. Keep the history.

FR-14 / Freshness and updates

Keep connectivity and lifecycle separate. Label stale work with last-seen time rather than silently clearing ownership. A later prompt can continue an existing task, revise its scope, or start a new one. The user can correct mistaken grouping. Pin the supported PO behavior to one active top-level task per session.

FR-15 / Activity presentation

Group noisy tool events into meaningful bursts while preserving their expandable raw records. Filters include teammate, session, event type, and task status. Use a stable timeline; do not move a card under the pointer or steal scroll position as events arrive.

Acceptance: changing a task's status is reflected in both clients; transcript statements such as "we might build this later" do not become active work; agent-reported completion is not mislabeled as a tested or merged feature.

08 / Shared session viewer

Requirements FR-16 through FR-20.

FR-16 / Session header

Show the builder, agent type, project, task, branch, capture completeness, sharing state, and last received event. Include an explicit link back to the overview. A missing branch or unsupported message type is labeled unavailable, not guessed.

FR-17 / Readable transcript

Display user prompts and agent responses in a conversation layout with compact role labels and timestamps. Tool activity appears inline as collapsible rows, not giant chat messages. Default to collapsed successful calls; highlight failed calls without implying they represent a project failure.

FR-18 / Tool details

Expanded calls show name, sanitized arguments, result excerpt, execution status, and associated file paths. Respect truncation limits and display a "truncated" or "redacted" label when applicable. Retain the call/result correlation so delayed results attach to the correct call.

FR-19 / Navigation and evidence links

Support session-local text search, event-type filters, copy of sanitized content, and stable deep links to a message or tool event. A coordination alert opens the exact supporting event and highlights it. If evidence was removed or access changed, show "Evidence no longer available" instead of cached private content.

FR-20 / Follow-live behavior

Follow the newest activity only while the user is already at the end of the session. Scrolling up pauses following and shows a new-activity counter. Provide a deliberate "Jump to latest" control. Fetch initial history in pages and recover missed events by durable cursor after reconnect.

State	Required UX
No messages yet	Explain that the connection is ready and waiting for the first shared turn.
Partial capture	State which event categories are missing and why.
Sharing paused	Show the last shared event and a clear paused indicator.
Content deleted	Remove cached content and retain only a safe tombstone where needed.

Acceptance: selecting a cited event lands on the correct record; scrolling history does not jump when new events arrive; a large tool result cannot freeze the page; redacted values never reappear through copy or search.

09 / Before-you-build checks

Requirements FR-21 through FR-25.

FR-21 / When a check runs

Run a preflight when a shared session submits a new implementation request or materially changes the current task scope. Classify obvious conversation-only turns separately. A follow-up on the same unchanged task should reuse current evidence where safe, not spam the user with the same warning.

FR-22 / Two independent findings

Evaluate **code evidence** and **team-work relationships** independently. A feature can exist on main while a teammate is changing it. Multiple findings may coexist; never force the answer into one mutually exclusive "exists or in progress" label.

Finding	Message and next action
Implementation evidence found	Show what code appears relevant, its source version, and scope. Offer "View implementation" or "Use existing context."
Likely duplicate task	Show owner, intended outcome, freshness, and evidence. Offer "Coordinate," "Revise my task," or "Proceed separately."
Related or complementary work	Show dependency or interface context. Allow normal progress; do not treat similarity as a collision.
No relevant evidence found	State the coverage checked and allow progress. Never say the feature definitely does not exist.
Uncertain / unavailable	Explain missing context, stale index, provider failure, or ambiguity. Allow an explicit retry or continuation without a completed check.

FR-23 / Evidence-first explanation

Every nontrivial warning includes cited task or source IDs, a brief reason, the evidence timestamp, and coverage limitations. Do not present raw model confidence as a calibrated probability that duplication is real.

FR-24 and FR-25 / Human decisions and interruption

Default to advisory checks. Offer an explicit review-on-likely-duplicate setting for the demo project; it may stop that submitted prompt for review, not silently rewrite it. A timeout never becomes a green "clear" result. All decisions are recorded with actor, check revision, and reason where provided.

Acceptance: differently worded duplicate requests are detected in the demo fixture; complementary frontend/backend work is not blocked; provider failure is distinguishable from no match.

10 / Jev decision system

Implementation requirements for the classifier boundary.

TypeSafe documents Jev as a structured decision model over supplied state, including a Choice primitive. It does not automatically inspect a repository or teammates' sessions. The application supplies scoped evidence and translates provider responses into its own validated schema. [S02, S03]

Proposed decision pipeline

1. Normalize. Preserve the original request; extract only deterministic metadata or validated agent-supplied intent. **2. Retrieve.** Load authorized active intents and focused code evidence. **3. Compare.** Ask Jev bounded questions about specific candidate relationships. **4. Validate.** Enforce enums, evidence IDs, and coverage fields. **5. Apply policy.** Choose advisory, review, or unknown behavior outside the model.

Decision dimension	Application-owned labels
Request kind	implementation, conversation, continuation, unclear
Code coverage	present_evidence, partial_evidence, no_evidence, unknown
Relation to one task	duplicate_outcome, related_dependency, complementary, unrelated, uncertain
Evidence freshness	current, stale, missing; computed from recorded state, not guessed by Jev

Bounded context, not whole-history uploads

For a small project, include all short active-task intents within a hard cap before falling back to retrieval; this reduces keyword-only misses. Include at most a small, configurable set of code snippets with immutable IDs. TypeSafe notes context and information-selection limitations; retrieve and simplify rather than sending complete histories. [S04]

Explanation and confidence

Keep the initial explanation template-based: "This request appears to cover [scope] in [task]." Use validated evidence references and owner-supplied scope. Do not require Jev to generate long prose or invented citations. Persist model ID, prompt-policy version, candidate IDs, raw structured decision, and elapsed time for debugging with sensitive input logging disabled.

Failure and calibration

Invalid output, timeout, missing credentials, or unsupported schema produces unknown. Set review thresholds only after a labeled evaluation. Prefer precision for interrupting warnings; expose weaker matches as context. Log overrides to identify false positives, not as proof the user was wrong.

All enum names in this PRD are proposed product contracts, not copied TypeSafe API fields. Implement a provider adapter against the current official SDK/API rather than guessing request arguments.

11 / Repository evidence and coverage

Requirements FR-26 through FR-30.

FR-26 / Local-first approved inventory

The adapter inventories approved text files within the selected repository. For P0, support a small TypeScript/JavaScript project, record route/export names and relative paths, and produce bounded source chunks. Honor ignore rules and strict size caps. The server does not receive arbitrary filesystem access or clone private code with a broadly privileged token.

FR-27 / Versioned evidence

Each chunk has project and snapshot IDs, a relative path, commit SHA when available, branch, line range, content hash, captured time, and coverage status. Working-tree evidence additionally records its owner and dirty-state hash; it must not be labeled as merged or available to everyone.

FR-28 / Retrieval that matches the release scope

Start with route/export indexing, full-text matching, a small reviewed feature inventory, and direct inclusion of bounded active tasks. Code snippets must come from actual files, even if a feature description is curated for the demo. Embeddings and deeper syntax analysis are P1 options, not prerequisites for the core loop.

FR-29 / Refresh and invalidation

Refresh changed approved files after a completed write burst or explicit rescan, never from a simulated timer. Show snapshot version and last indexed time. A new branch, base commit, deletion, or content hash invalidates affected evidence. Never silently overwrite main's snapshot with another person's unmerged work.

FR-30 / Honest limitations

Evidence situation	Permitted wording
Relevant function found	"Related implementation found in this snapshot."
Only a planned contract exists	"Planned interface; implementation not yet observed."
Only branch-local code exists	"Exists on Sam's branch; not confirmed on main."
Search finds nothing	"No relevant evidence found in the indexed scope."
Snapshot is stale or partial	"This check may miss recent or excluded changes."

Acceptance: an existing demo feature produces a clickable real source reference; stale evidence is labeled; private or excluded files never enter retrieval; existing implementation is not treated as verified end-to-end functionality.

12 / Coordination and context handoff

Requirements FR-31 through FR-35.

FR-31 / Resolve a check

Provide four useful actions: use existing context, coordinate with a teammate, revise the task, and proceed separately. Each action records the check revision, the requesting builder, and the relevant task/source references. Dismissing a toast alone must not silently resolve the underlying check.

FR-32 / Minimal coordination thread

Keep a short project-scoped thread attached to the overlap, not a general chat app. Builders can propose a task boundary and attach an interface note. A person can change their own task immediately, but changing someone else's responsibility requires that person's acceptance. Show requested, accepted, rejected, and withdrawn states.

FR-33 / Structured context packet

Create a previewable packet containing the revised objective, related task IDs, owner, branch and snapshot identifiers, approved interface contract, cited excerpts, freshness, and explicit limitations. Label facts as implemented, planned, or user-confirmed. Keep the payload bounded and exclude private/unshared source content.

FR-34 / Deliver to the actual agent

The user approves the packet. The local adapter fetches it through an authenticated request and supplies it on the next supported prompt boundary. In PO, the browser offers a short resolution reference and an explicit "Copy continuation" action. The user resubmits in their terminal; the product does not pretend to have resumed an agent remotely.

FR-35 / Confirm the behavioral change

The next agent response and tool events should demonstrate the revised scope. The task card updates from proposed coordination to the builder's active responsibility. Do not mark a handoff successfully delivered until the adapter acknowledges receipt; do not equate receipt with compliance.

Example: Sam owns the saved-college API. Sri owns the shortlist UI. Both agents receive the agreed route and response shape; neither silently inherits the other's unfinished code.

Acceptance: a coordination decision survives reload; the packet is visible before submission; the agent receives the approved revision; stale or revoked references cannot be used; responsibility reassignment is never unilateral.

13 / Concurrency and lifecycle rules

Prevent coordination races without pretending to guarantee semantic uniqueness.

Register intent before checking

Create an idempotent proposed-intent record first, then retrieve candidates. Include recent proposed/checking intents as well as active ones; otherwise two simultaneous requests can each conclude that no teammate has started. Exclude the current intent and exact retries from their own candidate set.

Revision-aware checks

Maintain a per-project active-intent revision. A check records the revision it inspected. Before acknowledging start, compare that revision with current state in a short transaction. If it changed, recheck the affected candidates within the request budget or return "team activity changed; review pending." Do not hold a database lock during an external model request.

Scope changes and duplicate alerts

A task scope edit creates a new task revision and invalidates old resolutions. Deduplicate pairwise overlap alerts using the ordered task-ID pair and relevant revisions. Multiple prompts about one task should not create multiple identical warnings. Provide one-time overrides tied to the exact approved check and session.

Offline is not finished

Heartbeats indicate device freshness. A missing heartbeat marks evidence stale; it does not release a task as completed or authorize reassignment. The owner can explicitly pause or cancel. A later session may take over only through an attributable ownership change.

Completion and availability

An agent saying "done" moves a task to awaiting review. A builder can confirm done with a short note and available evidence. In PO, main-branch availability is derived from an explicitly refreshed main snapshot, not from a task status or an unverified merge claim.

Failure to avoid	Required behavior
Two agents start together	Both intents are durably visible; stale check revisions are detected.
Old resolution is replayed	Reject expired, consumed, wrong-session, or wrong-revision reference.
Network duplicates events	Use unique event IDs and acknowledge already-stored records.
Teammate disappears	Show stale/unknown status; do not invent completion.

Semantic matching remains probabilistic. Database transactions protect state consistency; they do not prove that two requests are semantically distinct.

14 / Information architecture

Few destinations, deep context, no sprawling navigation.

Route / destination	Purpose and principal content
/	Concise product introduction; sign in; a clearly labeled sample workspace, only if time remains.
/onboarding	Create/join project, pair a device, select sharing, verify first event.
/w/[workspace]/p/[project]	Primary workspace: current work, builder-agent pairs, relevant activity, coordination inbox.
/.../sessions/[session]	Full session route; also opens as a context-preserving panel on desktop.
/.../checks/[check]	Comparison, evidence, code coverage, related tasks, resolution actions.
/.../tasks/[task]	Task scope, state, session links, decisions and coordination history.
/.../settings	Members, invitations, devices, sharing, retention and repository coverage.
/demo	Isolated sample/replay environment with unmistakable fixture labeling; never mixed with live project data.

Navigation structure

Use a compact left rail with project switcher, Workspace, Sessions, Coordination, and Settings. In the first release, Sessions and Coordination may be filtered views of the same workspace rather than entirely separate products. The project name, connection status, and primary context remain visible.

Information hierarchy

Lead with the current work and decisions. Put raw tool output behind an expandable row. Keep evidence one click from every claim. A task is the organizing object; a session is the conversation record; a check is a decision point. Do not use these labels interchangeably.

State persistence

Encode selected task/check/session and filters in navigable routes or query parameters. Preserve scroll and tab state when opening an evidence drawer. Browser back closes contextual detail before unexpectedly exiting the project. Deep links re-check authorization on every load.

Page design must cover loading, empty, populated, reconnecting, partial data, permission denied, error, and deleted-content states. An empty screen should explain the next useful action rather than display decorative sample metrics.

15 / Design direction and resources

A premium builder workspace, not a generic AI dashboard.

Visual direction: light-first, calm, precise, and information-rich without crowding. Use warm-neutral surfaces, dark ink, a restrained indigo action color, careful typography, and strong spatial grouping. The first impression should come from the real workspace, not a large gradient hero.

Resource	Role and boundary
UI/UX Pro Max	Use for visual-direction exploration, hierarchy, interaction critique, responsive behavior, and accessibility review. Verify the installed skill and follow its actual instructions. [S08]
shadcn/ui	Base component vocabulary for buttons, dialogs, drawers, forms, tabs, menus and tooltips. Adapt to project tokens. [S09]
Magic UI	Optional, restrained motion for connection and state transitions; do not add decorative effects to the reading surface. [S10]
21st.dev	Selective inspiration or component source where it improves a real screen. Normalize styling and verify each license. [S11]
Saved design assets	Inspect the builder's actual saved GitHub resources and local skills before implementation. Exact repository URLs were not present in the retrieved prior PRD; do not invent or claim to have loaded them. [U02]

Proposed design tokens

Canvas **#F7F8FA**; surface **#FFFFFF**; text **#172033**; muted text **#667085**; border **#E2E6ED**; primary **#4F46E5**. Status colors always pair with an icon and text. Validate final color combinations rather than assuming these starting tokens guarantee contrast.

Use a single modern sans family such as Inter for UI and one mono family for code. Suggested scale: 28/34 page title, 20/28 section title, 15/22 main body, 13/18 metadata, 12/18 code. Base spacing: 4, 8, 12, 16, 24, 32 pixels. Card radius: 10-12 pixels; controls: 8 pixels. Shadows only for layered surfaces.

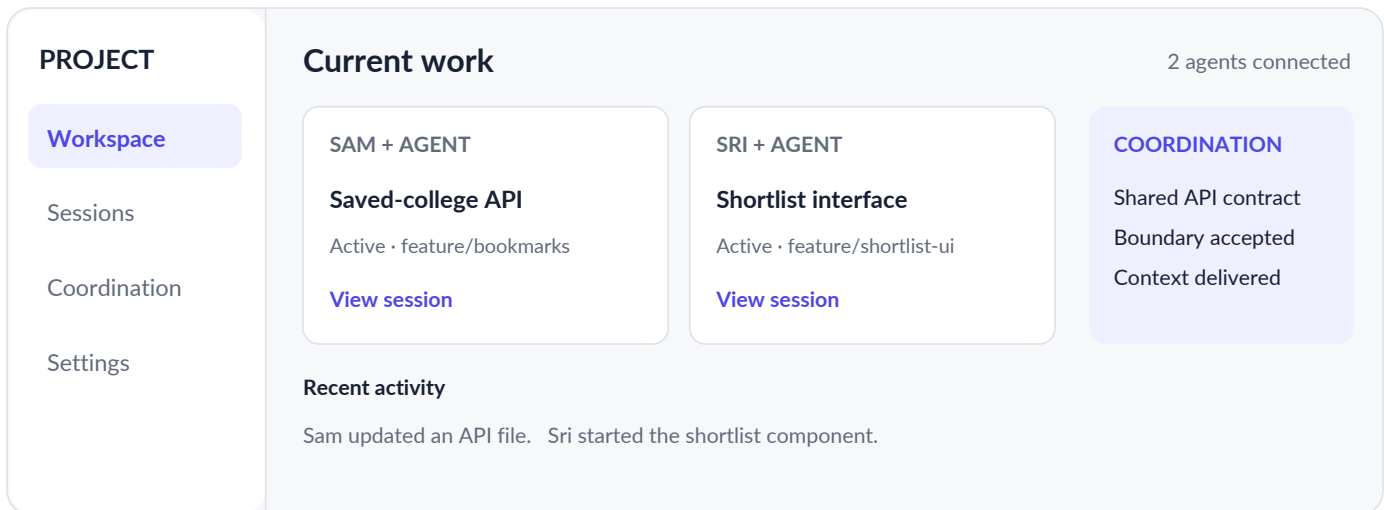
Design resource rule

Use the approved resources as a toolkit, not a checklist of visual effects. Every imported component must share the same tokens, density, focus behavior, and motion language. Record source and license in a small component-provenance file.

Avoid glowing borders everywhere, floating particle backgrounds, glassmorphism over transcripts, giant empty cards, fake activity counters, inconsistent icon families, and excessive pills.

16 / Workspace composition

The principal screen should explain the product in one glance.



A / Persistent orientation

The rail is approximately 216 pixels wide on desktop. The header shows project, repository, and factual connection health. Keep an obvious Connect agent action when no device is live. Do not bury it inside settings.

B / Current work is the hero

Each task card contains builder and agent identities, a concise scope title, lifecycle label, branch, last meaningful activity, and one or two related files. The primary action is View session. Related work and check status are small, explicit secondary signals.

C / Coordination deserves a clear place

A compact right-side panel shows unresolved checks and handoffs. A duplicate warning compares the intended outcomes and names the teammate; it should not just say "Conflict detected." Collapse the panel into a tab on narrower layouts.

D / Activity supports the story

Below current work, show a calm event stream with grouped tool activity. A new event has a brief entrance transition, not a repeated glow. Do not reorder active cards continuously according to every tool call.

Schematic above defines hierarchy, not a finished or implemented screen. The implementation team must apply the design system and real data before treating it as a visual acceptance reference.

17 / Critical screen specifications

Details that make the app feel complete.

Check detail / the main decision surface

Use a two-column comparison: "Your request" and "Related team work." Below, separate code findings from active-task findings. Show evidence excerpts in a bordered panel with source type, owner, branch/version, and freshness. Put the recommended next action in a stable footer; keep Proceed separately visible but secondary. The chosen action updates the actual check record.

Session detail / the evidence surface

Use a readable conversation column, a compact sticky header, and optional metadata side panel. A highlighted deep-linked event has a subtle border and a clear source anchor. Keep large JSON or code blocks collapsed, with horizontal scrolling confined to code. Search highlights matches and shows result count; never render unsafe HTML from transcripts.

Connect / the trust surface

A numbered progress component covers installation, pairing, repository choice, and first activity. The copy command button reports copy success only after the browser confirms it. Show a preview of redacted data and the destination project. Device pairing codes expire; errors preserve the user's current step and offer a specific retry.

Task and handoff / the collaboration surface

Show intended scope, current owner, state, sources, dependencies, and linked decisions. A coordination proposal visibly distinguishes the proposed split from the accepted split. A context preview includes a "Planned interface" label when no implementation is available. Do not make an accepted message look like a completed task.

Settings / control without clutter

Group members/invitations, devices, session sharing, and source coverage into tabs. Risky actions use clear confirmation text naming the affected item. Pause sharing is reversible; delete shared content is separate. Show what has already been shared and the retention window.

Interaction	Required result
Copy evidence link	Copies a stable authorized deep link, with confirmation.
Open source	Opens available versioned evidence; never guesses a local path on another machine.
Resolve check	Persists immediately; optimistic UI rolls back visibly on failure.
Open another session	Maintains project context and browser history.

18 / Interaction, accessibility and polish

Quality requirements DQ-01 through DQ-07.

DQ-01 / Responsive layouts

Design and review at 1440, 1280, 1024, 768, and 390 pixels. Desktop may use overview plus inspector. Tablet switches to one main column and a drawer. Mobile uses stacked tasks, a compact navigation control, and full-page detail. No essential action depends on hover. Code may scroll horizontally; the page must not.

DQ-02 / Accessibility target

Target WCAG 2.2 AA with semantic structure, visible focus, sufficient contrast, keyboard operation, labeled inputs, accessible dialogs, and meaningful status announcements. This is a product target, not a compliance certification. Perform automated and manual checks. [S12]

DQ-03 / Motion and feedback

Use approximately 120-180 ms for control feedback and 180-240 ms for panel transitions. Respect reduced-motion preferences. Reserve motion for cause and effect: pairing succeeded, a check arrived, or a resolution changed state. Do not animate the entire transcript on each update.

DQ-04 / Density and loading

Main task content remains readable at default browser zoom. Use skeletons matching final layout, not a full-screen spinner for every request. Keep controls stable as data loads. Provide visible stale/loading states for checks and connections instead of showing old data as current.

DQ-05 / Exact microcopy examples

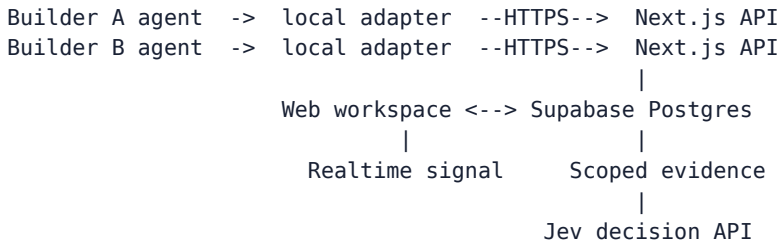
Situation	Preferred copy
No match	No relevant overlap found in the evidence checked.
Possible duplicate	Sam is working on a similar outcome. Review their scope before starting.
Provider failure	The coordination check is unavailable. You can retry or continue without it.
Branch-only evidence	Found on Sam's branch; not confirmed on main.
Sharing paused	New activity is not being shared. Previously shared content remains until deleted.

DQ-06 and DQ-07 / Final review

Test all keyboard paths, modal focus return, long names, empty repositories, long tool arguments, 200% zoom, and disconnected sessions. No placeholder CTAs, dead filters, unlabeled icons, false "live" badges, or notification-only success for actions that failed on the server.

19 / Technical architecture

Separate local code execution from shared coordination.



Proposed stack

Use **Next.js App Router + TypeScript** for the web application and short-lived API endpoints; Tailwind and the chosen component system for UI; Supabase Auth and Postgres for identity and durable project data; Supabase Realtime for change signals; and Vercel for preview and production deployment. The App Router is documented in current Next.js guidance. [S05]

Local adapter responsibilities

Run alongside the builder's agent. Own consent, repository binding, redaction, local capture, event buffering, source snapshots, preflight invocation, and context delivery. It is the only component that reads approved local files. It does not accept arbitrary remote shell commands.

Cloud responsibilities

Authenticate and authorize requests; ingest idempotent events; store tasks and decisions; retrieve scoped evidence; call Jev; serve the workspace; and publish minimal change notifications. The server never executes the builder's code. Use durable rows as truth, not transient socket messages.

Background work and deployment

Keep source extraction local and cloud requests bounded. Do not depend on a long-running process inside a Vercel request: function limits exist and must be checked for the chosen deployment configuration. [S06] Defer an external job worker until asynchronous indexing or scale makes it necessary.

Realtime security

Use private project channels, publish only resource IDs and revisions where practical, and fetch sensitive content through an authorized endpoint. Supabase documents separate Realtime authorization behavior; database row policies alone are not a complete channel design. [S07]

No privileged infrastructure mutations are requested by this PRD. Pin actual dependency versions at build start, commit lockfiles, and verify provider documentation again when implementing.

20 / Data model

A compact schema with explicit ownership and evidence lineage.

Entity	Important fields and purpose
organizations / memberships	ID, name; organization_id, user_id, owner/member role, joined_at. Unique membership.
projects / invitations	Organization, name, canonical repo binding; invite token hash, role, expiry, consumed_at.
devices	User/project scope, token hash, agent + adapter version, allowed repo identity, last_seen, revoked_at.
sessions	Project/user/device, source session ID, agent, branch, sharing state, capture capabilities, start/end time.
events	Project/session, source_event_id, sequence, kind, occurred_at, received_at, redacted payload, tool correlation.
tasks / task_revisions	Owner/session, current state, source request, title, intent, branch; immutable scope revisions.
source_snapshots / chunks	Project, owner if local, branch, commit/dirty hash, coverage; relative path, line range, content hash, approved content.
checks / findings	Task revision, evidence snapshot, active-set revision, policy/model version, status; candidate IDs, relation, evidence references.
resolutions / handoffs	Check/task revisions, actor, action, accepted scope, context packet, delivery state, expiry and one-time consumption.
coordination_messages	Check/task thread, author, sanitized body, proposal/acceptance type, source references.
audit_events	Actor, project, action, target, time, outcome. No raw prompts, code or credentials.

Required constraints and indexes

Use UUIDs or equivalent unguessable IDs, UTC timestamps, foreign keys, and explicit project ownership on content records. Unique source event keys prevent duplication. Index project + task state, session + sequence, project + recency, and source snapshot + path. Reject invalid state transitions in the server, not just the interface.

Authorization and cleanup

All content queries enforce current project membership and sharing eligibility. Sensitive source, event, and handoff rows are never globally readable. Deletion propagates to derived search records, check excerpts, and context packets; safe tombstones preserve only minimal relationship integrity.

PO can store approved event payloads as bounded JSONB. Large attachment storage, embeddings, and a separate analytics warehouse are not required. Supabase row-level security is part of the authorization design. [S13]

21 / API surface

Proposed application contracts, not third-party endpoint claims.

Endpoint	Behavior
POST /api/invites; POST /api/invites/accept	Create or consume a scoped single-use invite. Owner-only creation.
POST /api/devices/pair; DELETE /api/devices/:id	Pair via expiring code; revoke a device and terminate future ingestion.
POST /api/agent/events	Validate and ingest a bounded batch; acknowledge accepted and already-present event IDs.
POST /api/agent/snapshots	Ingest approved versioned source chunks with manifest and content hashes.
POST /api/agent/preflight	Register intent, retrieve authorized evidence, run comparison, return decision and check ID.
POST /api/checks/:id/resolve	Create a revision-bound resolution; reject stale or unauthorized writes.
GET /api/agent/resolutions/:id	Fetch approved context for the bound device/session; validate expiry and scope.
POST /api/agent/resolutions/:id/ack	Atomically acknowledge consumption/delivery, with retry-safe request ID.
PATCH /api/tasks/:id	Owner-scoped task status or scope change using expected revision.
GET /api/projects/:id/activity	Fetch authorized activity after a cursor; supports recovery after missed realtime signals.

Contract requirements

Use schema validation for every input and output. All mutations accept an idempotency key or naturally unique source identifier. Never trust client-supplied user/project ownership without checking it against authenticated scope. Return typed error codes, safe human-readable messages, and request IDs.

Limits and errors

Proposed starting limits: 50 events per ingest batch, 16 KB of sanitized payload per event, 512 KB batch body, and explicit source-chunk caps. These are application defaults to test and tune. Use 400 for malformed data, 401/403 for auth failures, 409 for stale revisions, 413 for limits, and 429 for rate limits; retain useful retry instructions.

Sensitive reads should be non-cacheable in shared intermediaries. Browser endpoints use validated user sessions and CSRF/origin controls; adapter endpoints use device credentials. Neither accepts a project ID as proof of permission.

22 / Example data contracts

Normalized product payloads; illustrative IDs and content only.

Event envelope

```
{
  "schema_version": 1,
  "event_id": "evt_demo_014",
  "session_id": "ses_sam",
  "source_sequence": 14,
  "kind": "tool.completed",
  "occurred_at": "2026-10-03T18:00:00Z",
  "payload": {
    "tool_name": "Write",
    "tool_call_id": "call_009",
    "relative_paths": ["src/api/bookmarks.ts"],
    "status": "success",
    "result_excerpt": "Created file",
    "redacted": true,
    "truncated": false
  }
}
```

Preflight result

```
{
  "check_id": "chk_demo_02",
  "task_revision": 1,
  "active_set_revision": 18,
  "status": "review_suggested",
  "code_coverage": "no_evidence",
  "findings": [{
    "candidate_task_id": "task_sam_api",
    "relation": "duplicate_outcome",
    "evidence_ids": ["evt_demo_003"],
    "freshness": "current"
  }],
  "limitations": ["Only approved project evidence checked"],
  "allowed_actions": ["coordinate", "revise", "proceed"]
}
```

Authenticate device and derive project/user scope server-side. The example omits credential headers intentionally. Adapter capability metadata must distinguish events that are observed from those inferred. Provider output is normalized before it enters this response.

23 / Adapter execution and continuation

The key implementation risk must be proven before the dashboard expands.

Connection and local setup

Create a project-scoped adapter configuration outside tracked secrets. Pair through an expiring device code, store credentials in an OS credential store where supported, and otherwise use a protected local file excluded from Git. Validate repository-root binding before reading files; do not follow symlinks outside the approved root.

Prompt flow

On submission, identify the session and task revision, prepare a sanitized request, and call preflight within the time budget. When the check is clear or advisory, return bounded team context. When opt-in review is required, stop that submitted prompt with a concise explanation and a link to the decision screen. Preserve the user's text for an explicit retry.

Claude Code's prompt hook can add context but does not replace the submitted prompt. Implement revised work as a user-approved continuation rather than assuming a browser action rewrites a running turn. [S01]

Tool and message capture

Map supported tool events to the normalized envelope and pair input/result IDs. Read only allowed user-visible message records through a tested parser or supported message event. Maintain parser fixtures for the pinned agent version. Capture gaps must be visible. Keep optional or unstable event sources disabled until verified.

Resolution delivery

The builder resolves a check in the web app and copies a continuation reference. On resubmission, the adapter validates the reference, task/check revision, actor, scope, expiry, and one-time status. It injects the approved packet alongside the user's revised objective and acknowledges successful delivery. On invalid reference, do not silently fall back to an old packet.

Failure behavior

A cloud outage must not corrupt or take over the local agent. Return an explicit unknown check result and follow the user's configured continue/review preference. Do not bypass the coding agent's native safety or permission controls. Stop sharing immediately when local consent is revoked; server revocation rejects further ingestion.

First build gate: prove a real prompt can be observed before relevant writes, a real tool event reaches a second client, and approved context reaches the real agent. If the chosen integration cannot do this, narrow the advertised support rather than faking the loop.

24 / Security and privacy

Trust is part of the minimum product, not post-demo cleanup.

Project isolation and credentials

Validate membership for every sensitive read, write, source fetch, and decision. Enable and test row-level security on exposed tables. Keep privileged database and provider keys server-only; never place them in public browser variables. Device tokens are hashed, project-bound, revocable, and minimally scoped. Supabase documents the row-policy foundation. [S13]

Local redaction and minimization

Exclude secrets and sensitive paths before queueing. Redact known token patterns, credentials in command arguments, and unnecessary absolute paths. Prefer metadata and brief excerpts over complete tool output. Pattern-based filtering is not a guarantee against all secrets: disclose its limits and use a synthetic, non-sensitive demo repository.

Untrusted content and model access

Treat prompts, transcripts, code comments, and tool output as untrusted data. Delimit them in classifier requests; never execute instructions found inside them. Validate output against fixed enums and source IDs. Send Jev only the authorized evidence necessary for a check and disclose that processing in the sharing flow.

Retention and deletion

Proposed default: retain shared content for 14 days in the prototype, configurable by the project owner; keep only minimal operational audit metadata longer. Users can pause sharing and delete their shared sessions separately. Delete derived excerpts and packets, clear browser caches, and document backup-retention limitations. Never promise instant deletion from previously exported or copied content.

Realtime and member removal

Use private channels and minimize broadcast payloads. On membership removal, revoke devices, deny reads immediately, and disconnect or refresh subscriptions. Fetching sensitive content must re-check current access even when a stale client still has a channel subscription. Show a permission-change state and clear previously rendered content.

Minimum security tests

Test cross-project IDs, direct evidence links, forged device scope, expired/replayed resolutions, invite reuse, oversized payloads, transcript HTML injection, source-path traversal, and secret-bearing command output. Raw prompts and source code must not enter analytics, error trackers, or general server logs.

The first release is a scoped collaboration prototype, not a certified enterprise security platform. Public onboarding of sensitive repositories should wait for a separate security review.

25 / Reliability, performance and operations

Initial engineering targets, not measured performance claims.

Target	Proposed P0 acceptance threshold
Workspace load	Usable primary content within 2.5 seconds on a normal development broadband connection. Measure deployed behavior.
Event freshness	95% of acknowledged events visible in another connected client within 2 seconds under the demo load.
Preflight	Target p95 at or below 5 seconds; stop waiting after a 6-second application deadline and return unknown.
Context delivery	Return a cached approved packet promptly; never perform a fresh large indexing job during resubmission.
Demo capacity	Two to five builders, ten recent sessions, and 5,000 bounded events in the seeded test project.
Reconnect	Recover durable events after the last acknowledged cursor without duplication or a full manual refresh.

Degraded-mode requirements

Realtime outage: show delayed status and poll authorized activity. Jev outage: retain visibility and surface unknown checks. Index unavailable: evaluate task overlap but label code coverage unavailable. Adapter offline: mark stale and retain prior evidence. Authentication expiry: request reauthentication without leaking a cached project to a different user.

Observability

Record request ID, project-safe identifiers, endpoint, latency, decision status, provider/model version, rate-limit events, ingestion failures, queue depth, and adapter capability state. Avoid raw payloads. Provide a small connection/debug panel usable during the demo; it is not a full observability product.

Budget and resources

Track model calls and estimated spend through actual provider usage. Set a configurable project budget and per-minute cap before live testing; warn and return unknown rather than silently exceeding the cap. Hosting and model prices must be checked at build time. No unverified free-tier or dollar-cost promise is made here.

Deployment hygiene

Separate development, preview, and production projects or credentials. Seed only disposable data. Apply tested migrations before release, run a smoke test, and document rollback. The adapter runs locally with its own release version; a frontend rollback must not invalidate its protocol without a compatibility message.

26 / Success metrics and classifier evaluation

Measure useful coordination, not the amount of surveillance.

Product success hypotheses

Activation means two real builders connect to one project and can inspect each other's intentionally shared activity. A successful coordination loop means a check finds relevant work, a builder makes a decision, approved context reaches the agent, and subsequent activity reflects the chosen scope. Count these outcomes, not raw transcript volume.

Measure	How to collect it
Time to shared visibility	Start at first login; stop when two authenticated sessions are visible. Report observed time, not an assumed gain.
Coordination completion	Count checks with a recorded resolution, acknowledged context delivery, and linked next task activity.
Warning usefulness	Review duplicate warnings with the builders; track accepted, revised, overridden, and incorrectly matched cases.
Perceived clarity	Ask testers to identify current owners and tasks without opening every transcript.
Failure transparency	Record whether testers can distinguish no match, stale evidence, provider failure, and paused sharing.

Jev evaluation set

Create at least 30 labeled scenarios before final tuning: six paraphrased duplicate pairs; six related-but-complementary pairs; five unrelated requests; five existing/partial implementation cases; four stale/branch-only cases; and four uncertain or missing-evidence cases. Keep a separate subset unseen during prompt tuning.

Evaluation targets

Aim for at least 90% precision on interrupting duplicate warnings and at least 80% duplicate recall on the small labeled set. These are provisional release targets, not validated real-world accuracy. Report the actual numerator/denominator and false-positive examples; small samples do not support broad claims.

Required ablations and edge cases

Test task-only evidence, code-only evidence, and both together. Include synonyms, a shared file with unrelated edits, similar outcomes in different files, an unfinished plan, private work, scope revisions, and simultaneous intents. Add adversarial instructions embedded in a transcript and confirm they do not change the output contract.

Analytics event names: device_connected, session_shared, check_started, check_completed, check_unavailable, evidence_opened, resolution_created, context_delivered, task_scope_changed. Payloads contain IDs and state, not raw conversations.

27 / Quality assurance matrix

Minimum test coverage before calling the app functional.

Test ID	Scenario and pass condition
QA-01	Two real accounts join via invite; outsider cannot read their project.
QA-02	Revoked/expired/used invite is rejected with the appropriate state.
QA-03	Two adapters stream distinct sessions; events show correct owners and order.
QA-04	Duplicate/reordered event deliveries do not duplicate transcript entries or mismatch tool results.
QA-05	Secret fixture and excluded files never appear in uploaded payloads, search, or logs.
QA-06	Pause and revocation stop future sharing; removed member loses data access.
QA-07	Paraphrased duplicate triggers a cited finding before implementation in the review-enabled demo.
QA-08	Frontend/backend complementary work returns context without a false blocking conflict.
QA-09	Existing code opens the correct versioned source; branch-only evidence is labeled.
QA-10	No match and model timeout are distinct; timeout never becomes a clear result.
QA-11	Accepted context packet reaches the actual agent and affects its subsequent scope.
QA-12	Stale, replayed, wrong-session or unauthorized resolution is rejected.
QA-13	Concurrent intents cannot both consume a known-stale active-set revision unnoticed.
QA-14	Reconnect recovers missed events and preserves the user's transcript scroll.
QA-15	Keyboard-only path covers invite, connect, check, evidence, resolution and settings.
QA-16	390px layout, 200% zoom, reduced motion, long names and large tool results remain usable.
QA-17	All primary UI actions persist real state; refresh does not undo successful changes.
QA-18	Deployed production smoke test passes with two machines or isolated real sessions.

Test layers

Unit-test policy, normalization, redaction, revision checks, and state transitions. Integration-test authorization and provider adapters with fixtures. End-to-end-test the live workflow with a small controlled repository. Use browser accessibility checks plus manual keyboard and screen-reader inspection.

A mock provider is useful in automated tests, but the main demo must identify any replay or fixture mode clearly. Passing a mock suite is not evidence that the live agent integration works.

28 / Implementation plan and build gates

Build the difficult connection first; polish the proven workflow.

Gate	Deliverable and exit criterion
G0 / integration spike	A real prompt is intercepted; one tool call reaches a second client; approved context returns to the real agent. Verify Jev credentials and one structured decision. Do this first.
G1 / foundations	Auth, project membership, device pairing, schema, RLS and invitation flow work. Two users join and an outsider is denied.
G2 / visible work	Event ingestion, shared sessions, task state and overview use live durable data. Pause, reconnect and deduplication tests pass.
G3 / useful checks	Task and code evidence retrieval, Jev adapter, five finding states and evidence links work. Baseline evaluation cases pass.
G4 / coordination loop	Resolution, scoped context preview, explicit continuation and agent acknowledgement work. Revised task activity is observable.
G5 / design and robustness	All critical states use the design system; responsive, accessibility, security and live-demo tests pass.
G6 / release rehearsal	Production deployment, two-user smoke test, measured demo, reset procedure and labeled fallback recording are ready.

Suggested parallel workstreams

Builder A: adapter, capture, preflight and continuation. **Builder B:** schema, auth, ingestion, evidence and Jev policy. **Builder C:** design system, workspace, session/check surfaces and end-to-end polish. With two builders, combine backend and integration work and defer all P1 features.

Planning estimate, not a commitment

Reserve roughly 60-100 focused person-hours for the scoped prototype, integration testing, and polish, assuming the stack is familiar and credentials are available. This is an engineering estimate, not a verified hackathon schedule. G0 results should replace it with a team-specific estimate. A single-day event may require a narrower supported flow.

Cut order when time is tight

Cut marketing polish, general search, automatic summaries, extra adapters, rich PR sync, and elaborate analytics first. Do not cut the real agent integration, explicit sharing, evidence, meaningful unknown states, the handoff loop, or basic authorization.

29 / Live demo runbook

A three-minute story with a visible before and after.

Time	Action and what the audience sees
0:00-0:25	Introduce the shared project and two real builder-agent pairs. "Our agents currently work in separate conversations. This gives them shared project awareness."
0:25-0:55	Sam asks for the saved-college API. The task appears, and actual tool activity confirms work has started. Open one readable session excerpt.
0:55-1:25	Sri asks for the same outcome using "personal shortlist" wording. Before implementation, a check names Sam's overlapping task and exposes the supporting prompt.
1:25-2:00	Sri chooses coordination, scopes their task to the shortlist UI, and approves the planned API contract. Sam accepts any proposed change to Sam's responsibility.
2:00-2:35	Sri explicitly resubmits with the approved context. The agent acknowledges the shared boundary and starts real UI work. Show the two complementary task cards.
2:35-3:00	A judge submits a paraphrase or an unrelated change. Show that the response is evidence-dependent. End on the two tasks and the recorded handoff, not a log wall.

Prepared fixture

Use a small college-search demo repository with a visible existing search feature and an unimplemented shortlist feature. Keep two isolated branches/worktrees, two user accounts, clean credentials, and a reviewed example API contract. Any preloaded history is marked sample history; incoming agent events are genuinely live.

Evidence of the outcome

Capture timestamps for prompt submission, check result, first relevant edit, handoff delivery and the next agent action. The UI should show intent and action, not claim that a few seconds of tool activity produced a finished production feature.

Rehearsal and fallback

Rehearse three times, including a provider timeout and adapter disconnect. Keep a clearly labeled recording of a successful real run and an isolated replay mode for network failure. Tell the audience when switching to replay. The reset script only touches the disposable demo project and never real user data.

Suggested closing line: "Each builder keeps their agent. Now the whole team knows what is already built, what is underway, and how to build together."

30 / Definition of done

The release is complete only when the product loop is real.

Functional release gate

Two authenticated builders can join the same project, pair real supported agents, intentionally share activity, inspect each other's visible transcripts and tool calls, and see current tasks. A new implementation request is checked against both approved code evidence and current team work. The check cites evidence and distinguishes duplication, related work, missing evidence, and failure.

Coordination release gate

A builder can revise the task, agree a boundary where needed, preview approved context, deliver it to the real agent, and observe subsequent behavior consistent with the choice. The decision and task revisions persist across refresh. A browser button never falsely claims to have resumed an unsupported agent.

Design release gate

All primary screens use one token system and the chosen component language. Workspace, session, check, connection and settings views are polished at desktop and mobile sizes. Loading, empty, stale, error, permission, redaction and deletion states exist. Keyboard navigation and reduced-motion behavior work. No dead buttons or fake live metrics remain.

Trust and deployment release gate

Project isolation, device revocation, local filtering, explicit sharing, and source-access tests pass. Secrets are absent from client bundles and observability payloads. Deployed endpoints and two-user flow are smoke-tested. Setup instructions, seed/reset scripts, known limitations and protocol versions are documented.

Deliverables from implementation

Artifact	Required contents
Application repository	Web app, local adapter, shared schemas, migrations, tests, seed data and lockfiles.
Operational README	Install, configure, connect two users, run tests, deploy, reset demo and troubleshoot failures.
Design record	Tokens, component provenance, critical screen states and responsive review captures.
Demo package	Live script, disposable repository setup, measured rehearsal notes and labeled backup recording.

Do not declare the project done because the UI looks finished. Do not declare it done because the backend works without a usable collaboration experience.

31 / Risks, open decisions and roadmap

Keep assumptions visible while the team starts building.

Risk	Mitigation
Agent integration differs by version	Pin and test the first adapter at G0; publish a capability matrix and explicit unsupported states.
False duplicate warnings annoy builders	Optimize interruption precision; show evidence; allow revision and override; suppress duplicate alerts.
Code retrieval misses semantics	Bound the demo repository, include active intents directly, report coverage and uncertainty.
Private or stale context leaks	Enforce consent and project access at capture, retrieval, delivery and rendering; expire derived packets.
Product looks like an existing feed tool	Emphasize the complete outcome-level coordination loop; validate differentiation without claiming uniqueness.
Beautiful UI hides a nonfunctional demo	Implement live data and actual agent continuation before final motion or visual effects.

Decisions needed during kickoff

Confirm product name, participating builders, actual hackathon window/rules, the exact coding-agent version, access to Jev, and the approved demo repository. Confirm whether the team accepts the proposed light-first visual direction and retention defaults. These choices should not delay building the G0 integration spike.

Roadmap after a successful demo

First: improve task extraction, retrieval quality, evidence coverage, and onboarding based on real team use. **Then:** add a second adapter and shared interface-contract workflows. **Later:** repository/PR integrations, multiple projects, agent-to-agent coordination requests, and organizational controls. Do not expand into remote execution or automatic merges without a separate product and security review.

What would invalidate the current approach

Revisit the product when warnings are usually ignored, builders must constantly maintain task state, or shared context does not change agent behavior. Those results suggest the coordination experience or evidence model is wrong, even if the transcript viewer is attractive.

The goal is a strong collaboration app for builders. Duplicate detection is the initial useful trigger; the long-term product is shared project understanding and convenient coordination.

32 / Builder kickoff brief

A compact instruction set for the implementation agent and team.

Build the smallest beautiful version of the whole loop: connect, see work, check intent, inspect evidence, coordinate, continue.

Start here

Read this PRD and existing repository instructions. Inspect the actual installed design skills and approved assets; do not invent skill paths. Record assumptions and selected versions. Build GO before creating a large dashboard. Use one supported agent, two real users, and one small shared repository.

Implementation order

Define shared schemas and state transitions; establish authorized ingestion; prove prompt interception and context delivery; build the workspace shell and live session view; add explicit tasks and scoped evidence; integrate Jev behind a typed adapter; implement resolution and continuation; then complete all visual states and release tests.

Non-negotiable implementation rules

Never hard-code a successful live check. Never display fabricated tool calls as real activity. Never infer identity from a client-supplied user ID. Never upload all local transcripts by default. Never call unavailable evidence "clear." Never treat a model's confidence as ground truth. Never silently rewrite a user's task or someone else's ownership.

Design execution

Use UI/UX Pro Max for direction and critique; shadcn/ui for the base; optional Magic UI and 21st.dev elements where useful; and actual approved saved assets. Adapt everything to the same tokens. Spend the design effort on the workspace, check comparison, evidence viewer, onboarding, and state transitions - not an elaborate marketing homepage.

When an integration assumption fails

Stop expanding scope, document the actual limitation, and choose a supported narrower interaction. An explicit continuation copied to the agent is acceptable; falsely claiming autonomous resume is not. A small indexed repository is acceptable; pretending to understand unindexed code is not.

Final review question

Can an unfamiliar person watch two builders use their real agents and understand, without reading logs, how the app prevented repeated work and helped them collaborate? If not, improve the workflow before adding another feature.

This is a product requirements document, not an implementation-completion report. No repository, deployment, account, or external service has been modified by creating this PDF.

33 / Sources and specification status

Primary documentation checked on 3 October 2026.

Bracketed source IDs refer to the references below. External documentation supports integration facts; product scope, thresholds, architecture choices, estimates, and interface contracts are proposed requirements, not claims that the app is already implemented. Source titles are clickable.

ID	Source and use
U01	Product brief and decisions in this conversation
U02	Prior saved college_compass_prd.md
S01	Anthropic / Claude Code hooks reference
S02	TypeSafe / State
S03	TypeSafe / Choice
S04	TypeSafe / Jev 1.13 jaggedness
S05	Next.js / App Router
S06	Vercel / Functions limits
S07	Supabase / Realtime authorization
S08	UI/UX Pro Max / official repository
S09	shadcn/ui / Introduction
S10	Magic UI / Introduction
S11	21st.dev
S12	W3C / WCAG 2.2
S13	Supabase / Row Level Security

The earlier saved PRD establishes design preferences, not the existence or current installation of every resource. Exact saved asset repositories still need a kickoff inventory. No exhaustive competitive analysis, performance validation, legal review, or trademark clearance is asserted.

Version 1.0 · Prepared for Sri · Product name not yet chosen. Update this specification after the integration spike and first full demo rehearsal.